

AI Teaching Webapp Spec Document

1. Summary

A web app that generates a **structured, time-boxed curriculum** for a subject the user wants to learn. It sits between "ask a chatbot anything" (no structure) and "fixed online course" (no flexibility): the user states their subject, experience, and time budget, and the system generates a course — placement check, lessons with content and quizzes, and a final exam — tailored to that envelope.

Three modes share one engine:

Mode	Pre-check	Lessons	Post-test	Tone
New Subject	Placement exam	Full, from fundamentals	Final exam (pass/fail)	Instructional
Review	Review quiz (find gaps)	Condensed, techniques-first	Test exam(s)	Refresher
Quick / Casual	None	Short, efficient	None	Casual

Modes are not separate products — they're a parameter set (`depth`, `include_placement`, `include_final`, `tone`, `fundamentals_vs_techniques`) fed to the same generation pipeline.

2. Goals & Non-Goals

Goals

- The user learns more about the chosen topic than they did before.
- Intake asks useful questions that produce a better schedule.
- Generated guides are followable and effective — not too thin, not bloated.
- A new user can complete a full course end to end, on a hosted, publicly reachable app.

Non-Goals (explicitly out of scope)

- Not a replacement for human teachers; not a content library.
 - Practice problems **must not** reveal the answer on request — graduated hints first. (This is a hard design requirement, see §6.4.)
 - No worksheet generation or assignment grading at launch.
 - Vague, off-track, or padded guides are a failure, not a feature.
-

3. Tech Stack (decided)

Layer	Choice	Notes
Frontend	Next.js (React)	App Router; SSR for landing/profile, client components for the lesson player.
Backend	Python + FastAPI	All business logic, generation orchestration, scoring.
Database	PostgreSQL via Neon	Serverless Postgres; JSONB for generated content.
Auth	Clerk	Free tier covers our scale; FastAPI verifies Clerk JWTs.
Hosting	Render	One Web Service for Next.js, one for FastAPI, plus a Background Worker (see §4.3).

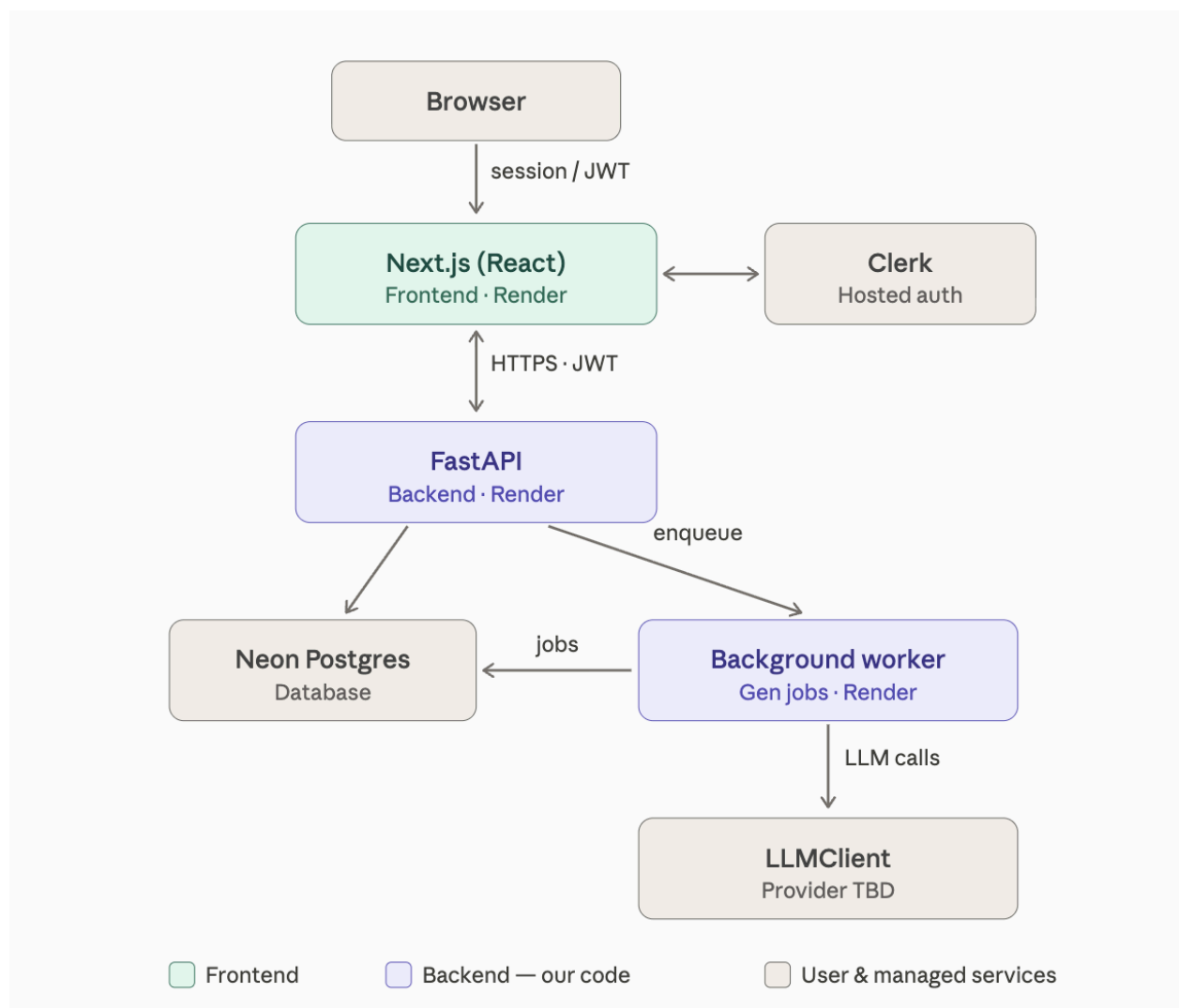
LLM
provider

Deferred

Hidden behind a thin **LLMClient** interface (see §6.1) so
the choice can be made later with zero rework.

4. Architecture

4.1 Topology



4.2 Auth flow (Clerk + split stack)

The frontend handles sign-in with Clerk's components. On every API call it sends the Clerk-issued JWT in the `Authorization` header. FastAPI validates that token against Clerk's public keys (JWKS) in a dependency and resolves the local `users` row. We never store passwords. A **Clerk webhook** (`POST /webhooks/clerk`) keeps our local `users` table in sync on user creation/deletion.

No app code ever handles credentials — Clerk owns that surface entirely.

4.3 Handling slow generation

Generating a course is a multi-step LLM pipeline that can take from tens of seconds to a few minutes — too long for a request/response cycle. Design:

- **Course generation = async job.** `POST /intake` creates the course row (`status=generating`), enqueues a job, and returns immediately with `course_id + job_id`. The frontend polls `GET /jobs/{id}` and shows progress.
- **Worker, not in-process.** Jobs run on a **Render Background Worker** reading from a `generation_jobs` table. This survives deploys/restarts (a FastAPI `BackgroundTask` would not). For v0.1 a background task is acceptable as a shortcut, but the worker is the target before v1.
- **Lesson content streams.** Individual lesson text is generated on first open and **streamed** token-by-token to the UI for a responsive feel, then persisted so it's never regenerated.

Open infra question: the worker/web split can coordinate purely through the `generation_jobs` table (simplest, no extra services) or via a Redis queue on Render. Recommendation: start table-only; add Redis only if we outgrow polling. Flagged in §10.

5. Data Model

users — *identity mirror of Clerk*

Column	Type	Notes
id	uuid	Primary key
clerk_user_id	text	Unique, not null
email	text	
created_at	timestampz	Default now()

courses — *one generated course = one learning envelope*

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK → users
subject	text	Not null
mode	text	Not null. new review quick
experience	text	beginner some confident ...
depth	text	shallow standard deep
total_weeks	int	...or a deadline
sessions_per_wk	int	
minutes_per_ses	int	
status	text	Not null. generating active completed
structure	jsonb	Generated outline (cached)
created_at	timestampz	Default now()

modules — “Topics”

Column	Type	Notes
id	uuid	Primary key
course_id	uuid	FK → courses
title	text	
ordinal	int	Order within the course

lessons — “Abilities within topics”

Column	Type	Notes
id	uuid	Primary key
module_id	uuid	FK → modules
title	text	
ordinal	int	Order within the module
content	jsonb	Null until generated; sections, data, graphs
status	text	Default 'available'. available completed
generated_at	timestampz	When content was generated

exams

Column	Type	Notes
id	uuid	Primary key
course_id	uuid	FK → courses
lesson_id	uuid	FK → lessons, nullable. Set for per-lesson quizzes

Column	Type	Notes
kind	text	Not null. placement lesson_quiz final review test
questions	jsonb	Not null. Items + rubric / answer key
created_at	timestampz	Default now()

attempts

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK → users
exam_id	uuid	FK → exams
answers	jsonb	
score	numeric	
passed	boolean	
archived_at	timestampz	Set on reset; preserves history for metrics
created_at	timestampz	Default now()

lesson_progress

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK → users
lesson_id	uuid	FK → lessons
status	text	in_progress completed
time_spent_sec	int	Default 0
last_viewed_at	timestampz	
archived_at	timestampz	Set on reset; row is no longer the active attempt
	constraint	<i>Unique (user_id, lesson_id) where archived_at is null — one active row per user + lesson</i>

schedule_items

Column	Type	Notes
id	uuid	Primary key
course_id	uuid	FK → courses
session_no	int	
planned_on	date	
lesson_ids	uuid[]	What to cover that session
est_minutes	int	

generation_jobs

Column	Type	Notes
id	uuid	Primary key
course_id	uuid	FK → courses
kind	text	structure schedule lesson exam
status	text	queued running done error
progress	int	Default 0. 0–100 for UI
error	text	Populated on failure
created_at	timestampz	Default now()
updated_at	timestampz	Default now()

events — *engagement, for metrics / streaks*

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK → users
kind	text	app_open lesson_view exam_submit ...
metadata	jsonb	
created_at	timestampz	Default now()

6. AI Generation Pipeline

6.1 Provider abstraction (because provider is deferred)

All model access goes through one interface so the provider decision stays open and swappable:

```
class LLMClient(Protocol):
```

```
    def generate(self, *, system: str, prompt: str,  
                 schema: type[BaseModel] | None) -> BaseModel | str: ...
```

- **One implementation at a time**, selected by env var. Switching providers = one new adapter, no business-logic changes.

- **All prompts live in one module** (`prompts/`), versioned, never inline.
- **Structured output is enforced:** every generation step returns JSON validated against a Pydantic model (via the provider's structured-output / tool-calling). On parse failure, retry once, then surface a job error.

6.2 The pipeline (New Subject mode)

intake → course structure → placement exam → (score) → plan

→ schedule → [per lesson: content → practice problems]

→ final exam

1. **Course structure** — from subject + experience + depth, produce Subject → Modules(Topics) → Lessons(Abilities). Cached in `courses.structure` and expanded into `modules/lessons` rows.
2. **Placement exam** — generated from the structure; scoring marks lessons known vs. partially known vs. needed.
3. **Condense** — known lessons are pruned/marked optional and partially known lessons given less priority so the plan fits the time budget.
4. **Schedule** — distribute remaining lessons across `sessions_per_wk × minutes_per_ses` into `schedule_items`.
5. **Lesson content** — generated lazily on first open, streamed, then persisted.
6. **Practice problems & exams** — generated per lesson / per course.

Review and Quick modes run the same pipeline with steps toggled off (no placement/final in Quick; review-quiz instead of placement, techniques-first content in Review).

6.3 Auto-grading strategy

Start with **multiple-choice and short structured answers** — reliably auto-gradable against the stored answer key, no model call needed to grade. Free-text/essay grading (which needs an LLM rubric pass) is deferred; it's the unreliable part and not worth the launch risk.

6.4 "Hints, not answers" (hard requirement)

The tutor endpoint (`POST /tutor/hint`) returns **graduated hints** keyed to a hint level that escalates only on repeated asks — it never returns the full solution. The system prompt enforcing this lives server-side and is never exposed to the client, and user input is treated as untrusted (see §10, abuse).

6.5 Resetting a course or lesson

A **reset** clears progress so the user can go through the same material again — it does **not** regenerate or change the content (`lessons.content` and `exams.questions` are untouched). "Back to default" means a fresh slate over the existing course.

Mechanism is the same at both scopes — we *archive* rather than delete, so streaks, pass rates, and time-on-task stay intact:

- **Course reset** — stamp `archived_at = now()` on every active `lesson_progress` and `attempt` row for the course, set all its `lessons.status` back to `available`, and set `courses.status` back to `active`.
- **Lesson reset** — same, scoped to a single `lesson_id`: archive that lesson's active progress and its quiz attempts, set the lesson back to `available`.

The app only ever reads non-archived rows as "current," so a reset instantly looks fresh while history is preserved underneath. (A future variant could *regenerate* exams on reset to defeat memorization — out of scope for now; default is reuse.)

7. API Surface (FastAPI)

All routes require a valid Clerk JWT except the webhook. Shape, not final contract:

Method	Path	Purpose
POST	<code>/intake</code>	Create course + enqueue generation → <code>{course_id, job_id}</code>
GET	<code>/jobs/{id}</code>	Poll generation status/progress
GET	<code>/courses</code>	List the user's courses (profile / completed)
GET	<code>/courses/{id}</code>	Full course structure + progress

GET	<code>/lessons/{id}</code>	Lesson content (streams if generating)
POST	<code>/lessons/{id}/complete</code>	Mark lesson complete
POST	<code>/courses/{id}/reset</code>	Reset whole course to default (archive progress, restore lessons)
POST	<code>/lessons/{id}/reset</code>	Reset a single lesson to default
GET	<code>/exams/{id}</code>	Fetch exam questions (no answer key)
POST	<code>/exams/{id}/submit</code>	Submit answers → <code>{score, passed}</code>
POST	<code>/placement/{course_id}/submit</code>	Score placement → condense plan
POST	<code>/tutor/hint</code>	Graduated hint for a practice problem
POST	<code>/webhooks/clerk</code>	Sync user create/delete (no JWT; verify Clerk signature)

8. Phased Build Plan

Mapped to the brief's versions, with a foundation phase first because the team is new to building — get the plumbing working end-to-end before features.

Phase 0 — Foundation (de-risk first)

- Next.js + Clerk sign-in working; FastAPI verifying the JWT; Neon connected; both services deployed on Render with a "hello, authenticated user" round trip. Nothing AI yet. *Exit*: a logged-in user hits a protected API and gets their own data back, in production.

Phase 1 — v0.1 (skeleton course)

- Intake form (subject, time budget, experience).
- Generate course **structure** and **schedule**; render the outline.
- **generation_jobs** + polling UI.
- *Exit*: user submits intake and sees a real, scheduled outline.

Phase 2 — v1 (the actual product)

- Lesson **content** generation (streamed) + lesson player UI (content, data/graphs, quiz).
- **Practice problems** + auto-grading; the hints endpoint.
- **Exams** incl. placement and final; pass/fail; completed courses on profile.
- **Reset** controls — reset a whole course or an individual lesson to default (archive-based, content reused).
- *Exit*: a new user completes a full course end-to-end and it lands on their profile.

Phase 3 — v2 (modes)

- Review Mode and Quick Mode as parameter sets over the same engine.

9. Success Metrics & Instrumentation

Launch gates (binary, must be true to ship): full end-to-end completion works; app is hosted and publicly reachable.

Ongoing metrics (from the **events**, **attempts**, **lesson_progress** tables):

Metric	Definition
Final exam pass rate	$\text{passed finals} \div \text{finals attempted}$
Lessons completed	$\text{count, and completion rate} = \text{completed} \div \text{generated}$

Daily time on guide sum of `lesson_progress.time_spent_sec` per user per day

Streak consecutive days with an `app_open` event

Activation % of new users who finish intake → first generated course

10. Constraints, Risks & Open Questions

Constraints (from brief): team of 3, new to project building, minimal maintenance budget, target timeline of a few weeks.

Risks

- **LLM cost/latency** is the main variable cost and the main UX risk. Mitigated by persisting all generated content (never regenerate) and by potentially using a cheaper model for structure and a stronger one for content once the provider is chosen.
- **Content correctness** — generated lessons can be wrong (hallucination). Inherent to the approach; consider lightweight sourcing/citations post-launch. Off-track/unhelpful output is an explicit non-goal, so prompt quality and validation matter.
- **Abuse / prompt injection** — users may try to jailbreak the tutor into giving answers. Keep all system prompts server-side, treat user text as untrusted data, and rate-limit generation endpoints.
- **Worker durability** — long jobs must not die on deploy; hence the Background Worker over in-process tasks.

Open questions for the team

1. Worker coordination: `generation_jobs` table-only (recommended) vs. add Redis queue?
2. Time budget input: fixed deadline, or sessions-per-week + minutes-per-session, or both?
3. Do we gate lessons (must complete in order) or allow free navigation?
4. **Bring-your-own materials** (upload a textbook / notes / links and have the course built from *them*) — in scope, and if so which phase? Not currently supported. It's the largest single addition on the table: it needs file/object storage (Neon/Render disks aren't suited to large blobs — use an S3-compatible bucket such as Cloudflare R2), text extraction + chunking, embeddings, and a retrieval (RAG) step feeding relevant passages into the

generation prompts. The architecture absorbs it cleanly — store vectors in **pgvector on Neon** (no separate vector DB) and add an `embed()` method to `LLMClient` so the embedding provider stays swappable like the chat one — but it roughly doubles the backend surface, so the recommendation is to treat it as a **post-launch / v2 fast-follow**, not part of the few-week launch path.

11. Suggested Task Split (3 people)

Boundaries blur at the integration points — pair up in Phase 0 to get the contracts right.

- **Person A — Frontend.** Next.js app, Clerk integration, intake flow, lesson player (content + quiz + streaming), profile/completed-courses, job-polling UI.
- **Person B — Backend & data.** FastAPI, Neon schema/migrations, Clerk JWT verification + webhook, all endpoints, scoring, the job system and Background Worker.
- **Person C — Generation pipeline.** `LLMClient` abstraction, prompt design, the structured generation steps, auto-grading, and the graduated-hints logic.

Shared contracts (the API table in §7 and the JSON shapes for `structure/questions/lesson content`) should be agreed by all three up front — they're where the three areas meet.